

MEG-XL explicado paso a paso

Cómo un modelo de contexto largo lee palabras en señales cerebrales

Ricardo Díaz Peris

Dpto. de Sistemas Informáticos y Computación · UPV

Basado en Jayalath & Parker Jones (2026), *MEG-XL: Data-Efficient Brain-to-Text via Long-Context Pre-Training*, arXiv:2602.02494

Qué asumo y qué no asumo

Doy por sabido

- Qué es EEG y MEG, y qué mide cada uno.
- Filtrado, remuestreo, Nyquist, artefactos.
- Media, varianza, producto escalar, matrices.
- Que una señal cerebral tiene muy poca relación señal-ruido.

NO doy por sabido

- Qué es un *transformer*.
- Qué es la *atención*, un *token* o un *embedding*.
- Qué significa preentrenar o ajustar un modelo.
- Nada de jerga de aprendizaje profundo.

Cada término nuevo aparece **marcado** la primera vez y se explica con vocabulario de procesamiento de señal.

- 1 El problema: leer palabras del cerebro
- 2 Vocabulario mínimo: qué es un transformer
- 3 MEG-XL paso a paso
- 4 De la señal a la palabra: ajuste y resultados
- 5 Transferencia a EEG: el trabajo propio
- 6 Cierre

- 1 El problema: leer palabras del cerebro
- 2 Vocabulario mínimo: qué es un transformer
- 3 MEG-XL paso a paso
- 4 De la señal a la palabra: ajuste y resultados
- 5 Transferencia a EEG: el trabajo propio
- 6 Cierre

El objetivo: *brain-to-text* no invasivo



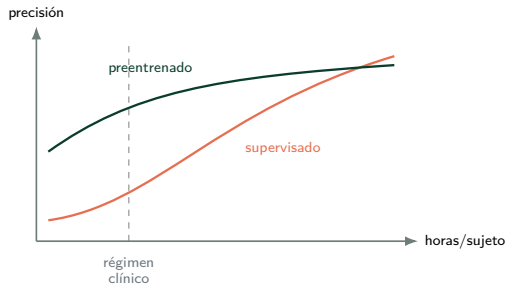
- La aplicación clínica objetivo: pacientes con parálisis o síndrome de enclaustramiento que **no pueden hablar**.
- No invasivo (MEG/EEG) evita la craneotomía, el tejido cicatricial y el reemplazo quirúrgico del implante.
- Precio a pagar: la relación señal-ruido es **órdenes de magnitud peor** que con electrodos intracraneales.

El cuello de botella no es el modelo: son los datos por sujeto

- El estado del arte supervisado (d'Ascoli et al., 2025) funciona, pero necesita **decenas de horas de grabación del mismo sujeto**.
- Sus análisis muestran que importan más las **horas por sujeto** que las horas totales.
- Un paciente con ELA no puede pasar 50 horas en un MEG.

La pregunta

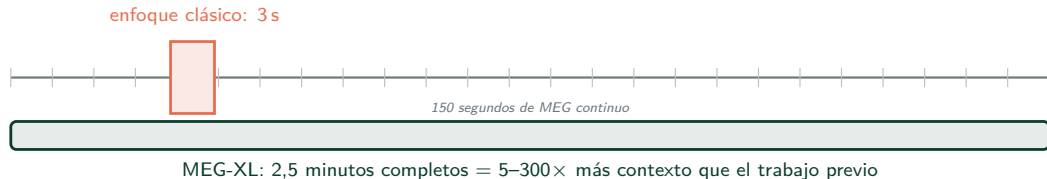
¿Se puede sustituir “muchas horas de *este* sujeto” por “muchas horas de *otros* sujetos”?



Esquema conceptual de la Figura 3 del artículo.

La segunda idea: nadie estaba mirando suficiente señal a la vez

- Los decodificadores clásicos recortan ventanas del **tamaño de la unidad que quieren decodificar**: 0,5–3 s para un fonema, sílaba o palabra.
- Eso asume implícitamente que **la información relevante es local**.
- Pero el habla tiene estructura a escalas mucho mayores: fonemas → sílabas → palabras → frases → discurso. Y la señal además arrastra estructura del sujeto, del ruido y del equipo.



Idea

Preentrenar un modelo con **muestras de 2,5 minutos de MEG continuo**, sin etiquetas, tapando trozos de señal y obligándolo a reconstruirlos. Después, ajustarlo a la tarea de decodificar palabras con muy pocos datos etiquetados.

150 s

contexto por muestra

191k

posiciones de entrada

20M

parámetros (vs 200M del supervisado)

~300 h

MEG de >800 sujetos

Resultado

Con el 13 % de los datos de ajuste: 47–57 % de acierto Top-10 frente al 20 % del azar, mientras que el resto de modelos fundacionales se quedan al nivel del azar.

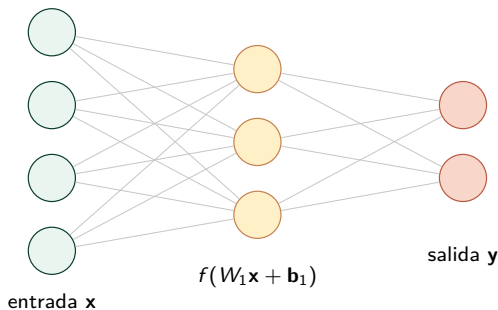
- 1 El problema: leer palabras del cerebro
- 2 Vocabulario mínimo: qué es un transformer
- 3 MEG-XL paso a paso
- 4 De la señal a la palabra: ajuste y resultados
- 5 Transferencia a EEG: el trabajo propio
- 6 Cierre

Una red neuronal es una función con mandos ajustables

Una **capa** de red neuronal hace exactamente esto:

$$\mathbf{y} = f(W\mathbf{x} + \mathbf{b})$$

- $\mathbf{x}$: vector de entrada (por ejemplo, 512 números).
- W : matriz de **pesos**. Es un banco de filtros lineales: cada fila calcula un producto escalar con la entrada.
- f : una no linealidad (aquí, **SELU**); sin ella, apilar capas no añadiría nada, porque la composición de funciones lineales es lineal.
- **Parámetros** = los números de W y $\mathbf{b}$. MEG-XL tiene 20 millones.



Vector de características (*embedding*)

Definición operativa

Un **embedding** es simplemente un **vector de d números reales** que representa un objeto (un instante de señal, una palabra, un sensor). Nada más.

Lo que ya conoces con otro nombre:

- El vector de potencias por banda $(\delta, \theta, \alpha, \beta, \gamma)$ de una época es un embedding de dimensión 5.
- Los coeficientes MFCC de una trama de voz son un embedding.
- La diferencia: aquí las componentes no las diseñas tú, las **aprende el modelo**.

Por qué importa la geometría:

- Si dos objetos son parecidos, sus vectores apuntan en direcciones parecidas.
- Se mide con la **similitud coseno**:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

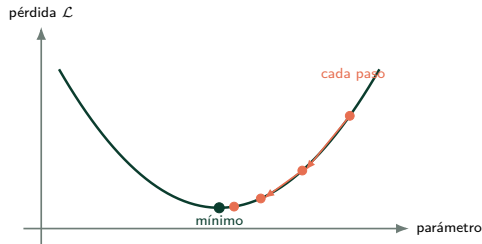
- Esto será literalmente el criterio de decisión final del sistema.

En MEG-XL, $d_{\text{model}} = 512$: cada instante de cada sensor se describe con 512 números.

Entrenar = bajar una pendiente

- 1 Se define una **función de pérdida** $\mathcal{L}$: un número que mide cuánto se equivoca el modelo.
- 2 Se calcula el **gradiente** $\partial\mathcal{L}/\partial W$: en qué dirección habría que mover cada parámetro para que la pérdida baje.
- 3 Se da un pasito en esa dirección:
 $W \leftarrow W - \eta \partial\mathcal{L}/\partial W$. La **tasa de aprendizaje** η es el tamaño del paso (10^{-4} en el preentrenamiento).
- 4 Se repite. MEG-XL da **35 000 pasos**, unas 12 horas en una GPU H100.

El optimizador concreto es **AdamW**: gradiente descendente con memoria del paso anterior y decaimiento de pesos.



Preentrenar y ajustar: la analogía del residente

Fase 1: preentrenamiento (*pre-training*)

- Se usan **muchísimos datos sin etiquetar**: 300 h de MEG de más de 800 sujetos, en reposo, tareas motoras y de lenguaje.
- No hay ninguna palabra que predecir todavía.
- El modelo aprende “cómo es una señal MEG normal”: qué ocurre a la vez en sensores vecinos, qué ritmos se encadenan, cómo evoluciona todo en el tiempo.

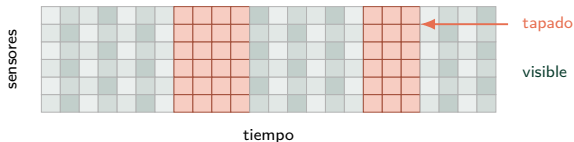
Fase 2: ajuste fino (*fine-tuning*)

- Se usan **pocos datos etiquetados**: grabaciones con las palabras marcadas en el tiempo.
- Se parte de los pesos ya aprendidos y se retocan suavemente (tasa de aprendizaje 10^{-5} , 100 veces menor).
- Es el residente que ya lleva años viendo trazados y ahora sólo aprende a marcar una patología concreta.

Hipótesis central del artículo: los priores aprendidos en la fase 1 sustituyen a las horas de grabación del sujeto en la fase 2.

Aprendizaje autosupervisado: tapar y adivinar

- **Supervisado:** te doy señal y te doy la respuesta correcta (la palabra). Caro: hay que grabar con estímulos anotados.
- **Autosupervisado:** la propia señal genera su etiqueta. Tapo un trozo y la etiqueta es *lo que había ahí*.
- Coste de etiquetado: cero. Se puede usar cualquier grabación MEG existente.
- Para acertar lo tapado, el modelo tiene que haber aprendido la estructura real de la señal.



El modelo debe reconstruir las columnas naranjas usando sólo lo verde.

¿Qué es un *token*?

En texto

Un **token** es un trocito de texto convertido en un **símbolo de un catálogo finito**. “pineapple” → token #4412 de un vocabulario de, digamos, 32 000 símbolos.

El problema en señales biológicas

El MEG es **continuo**. No hay ningún catálogo natural de símbolos. Hay que **fabricarlo**.

¿Por qué molestarse en discretizar? Dos razones muy concretas:

- 1 **Da un objetivo de entrenamiento bien definido.** Predecir un símbolo de entre 256 es un problema de clasificación con una pérdida estándar (entropía cruzada). Predecir un valor continuo obligaría a elegir una métrica de error en amplitud, dominada por el ruido.
- 2 **Comprime.** El tokenizador reduce el eje temporal por un factor $r = 12$. Sin esa compresión, la ventana de 150 s no cabe en la memoria de la GPU.

Atención (1/3): el problema que resuelve

Tenemos una secuencia de 625 vectores (uno por instante). Queremos que cada instante se **enriquezca con información de los demás**, pero:

- No sabemos de antemano **cuáles** son los instantes relevantes. Depende del contenido de la señal.
- Un filtro FIR clásico usa **pesos fijos** sobre una vecindad fija.

Atención, en una frase

Un **promedio ponderado** en el que los pesos se calculan a partir de los propios datos, en cada posición y en cada pasada.



Atención (2/3): consulta, clave y valor

Cada vector de entrada $\mathbf{h}_t$ se proyecta con tres matrices aprendidas y produce tres vectores distintos:

Consulta $\mathbf{q}_t = W_Q \mathbf{h}_t$

“Qué tipo de información estoy buscando.”

Clave $\mathbf{k}_t = W_K \mathbf{h}_t$

“Qué tipo de información ofrezco yo.”

Valor $\mathbf{v}_t = W_V \mathbf{h}_t$

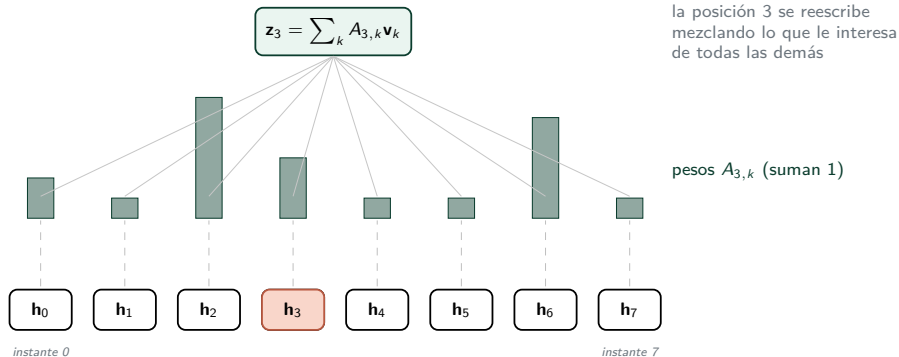
“El contenido que entrego si me eligen.”

El peso que la posición t da a la posición k es la **similitud entre la consulta de t y la clave de k** , normalizada:

$$A_{t,k} = \frac{\exp(\mathbf{q}_t \cdot \mathbf{k}_k / \sqrt{d})}{\sum_j \exp(\mathbf{q}_t \cdot \mathbf{k}_j / \sqrt{d})} \quad \Rightarrow \quad \mathbf{z}_t = \sum_k A_{t,k} \mathbf{v}_k$$

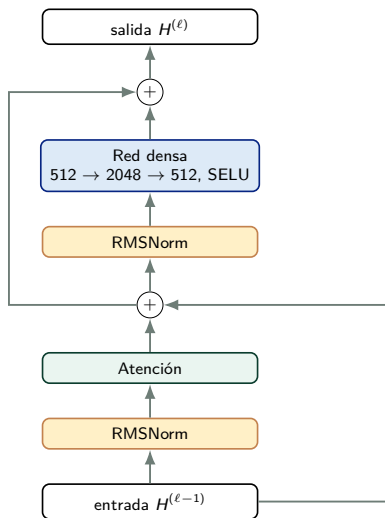
- El producto escalar $\mathbf{q}_t \cdot \mathbf{k}_k$ es un **detector de coincidencia**: grande si ambos vectores apuntan en la misma dirección. Es, literalmente, una correlación.
- La división por $\sqrt{d}$ evita que los productos escalares crezcan con la dimensión y saturen la exponencial.
- La **softmax** (exponencial normalizada) fuerza que los pesos sean positivos y sumen 1: es un promedio ponderado honesto.

Atención (3/3): qué sale de ahí, visualmente



- Se hace **en paralelo para todas las posiciones**, no secuencialmente. Por eso escala bien en GPU.
- **Múltiples cabezas** (8 en MEG-XL): se reparte la dimensión en 8 grupos y cada grupo aprende su propio criterio de similitud. Una cabeza puede mirar 200 ms atrás, otra 40 s.

El bloque transformer completo

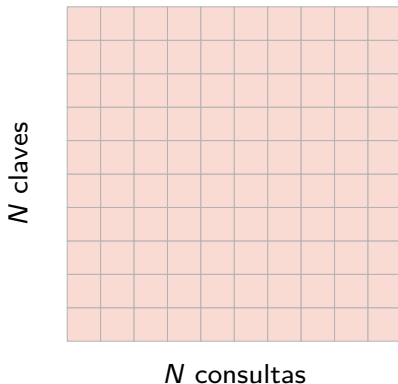


- **RMSNorm**: reescala cada vector para que su valor cuadrático medio sea 1. Es una normalización de amplitud, como estandarizar una época. Evita que las activaciones se disparen al apilar capas.
- **Pre-normalización**: se normaliza *antes* de cada sub-bloque. Hace el entrenamiento mucho más estable con muchas capas.
- **Conexión residual (+)**: la salida es *entrada* + *corrección*. Cada capa sólo tiene que aprender un **ajuste**, no rehacer la representación. Y el gradiente llega intacto a las capas bajas.
- **Red densa (*feed-forward*)**: dos capas que procesan cada posición **por separado**, con expansión $4\times$ ($512 \rightarrow 2048 \rightarrow 512$). La atención mezcla posiciones; esta red transforma cada una.
- MEG-XL apila $L = 8$ de estos bloques.

El talón de Aquiles: el coste es cuadrático

La matriz de pesos A tiene un peso **por cada par de posiciones**. Con N posiciones, eso es N^2 números.

N^2 pesos



Las cuentas reales de MEG-XL:

- $C = 306$ sensores, $T' = 625$ pasos temporales tras comprimir.
- Posiciones totales: $N = C \cdot T' = 191\,250$.
- $N^2 \approx 3,66 \times 10^{10}$ pesos...
- ...que en 16 bits ocuparían **≈ 73 GB** de memoria. **Por capa. Por cabeza.**

Conclusión

La atención completa sobre 2,5 minutos de MEG es **físicamente imposible** en el hardware actual. De ahí la atención *criss-cross*.

Posición: el modelo no sabe qué instante es cuál

Un detalle que sorprende

La atención es **invariante al orden**: si barajas las 625 posiciones, los pesos salen iguales. Es un conjunto, no una secuencia.

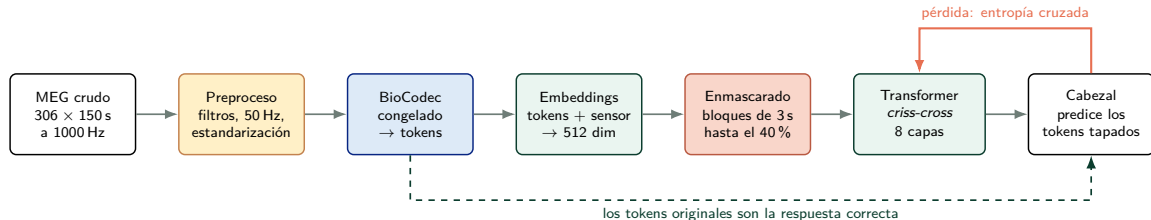
Hay que inyectar la información temporal explícitamente. MEG-XL usa **RoPE** (*rotary position embeddings*):

- Antes de calcular $\mathbf{q} \cdot \mathbf{k}$, cada vector se **rota un ángulo proporcional a su índice temporal**, por parejas de componentes.
- Al hacer el producto escalar entre la posición t y la k , las rotaciones se cancelan parcialmente y lo que queda depende de la **diferencia $t - k$** .
- Es decir: la similitud pasa a ser sensible a la **distancia temporal relativa**, no a la posición absoluta. Exactamente lo que interesa en una señal continua.

Solo se aplica en la atención **temporal**. En la atención espacial la posición la aporta otra cosa (las coordenadas 3D del sensor, dos diapositivas más adelante).

- 1 El problema: leer palabras del cerebro
- 2 Vocabulario mínimo: qué es un transformer
- 3 MEG-XL paso a paso**
- 4 De la señal a la palabra: ajuste y resultados
- 5 Transferencia a EEG: el trabajo propio
- 6 Cierre

El recorrido completo de una muestra



Vamos a recorrer cada caja, con las formas exactas de los tensores en cada punto.

Paso 0 — Preprocesado (el terreno conocido)

Filtrado y remuestreo

- Paso alto a **0,1 Hz**: elimina deriva lenta y artefactos del equipo.
- Paso bajo a **40 Hz**.
- Remuestreo a **50 Hz**.

Estandarización, por subsegmentos de 3 s

- 1 Corrección de línea base: restar la media por canal de los primeros **0,5 s** del subsegmento.
- 2 Escalar para que la mediana sea 0 y los cuartiles sean -1 y $+1$ (robusto frente a artefactos, a diferencia de z-score).
- 3 Recortar (*clamp*) al rango $[-5, 5]$.

¿0,1 Hz no mata el contexto largo?

No, y conviene tenerlo claro: el filtro elimina **componentes de frecuencia** por debajo de 0,1 Hz. No elimina las **dependencias estadísticas** entre respuestas separadas por minutos. Son dos cosas distintas: contenido espectral $\neq$ escala temporal de las dependencias.

¿Por qué estandarizar por subsegmentos de 3 s?

Porque en la fase de ajuste la entrada estará formada por ventanas de 3 s alineadas a palabras. Estandarizar igual en ambas fases hace que **el modelo vea la misma distribución** cuando se preentrena y cuando se ajusta.

Paso 0 (bis) — El elefante en la habitación: Nyquist

El problema

Paso bajo a 40 Hz y remuestreo a 50 Hz. Nyquist exige $f_s \geq 2f_{\text{corte}} = 80$ Hz. **Se está incumpliendo**, con riesgo de *aliasing*.

Por qué lo hacen igualmente: para poder comparar exactamente con d'Ascoli et al. (2025), que usan ese mismo preprocesado, y porque bajar a 50 Hz reduce a la mitad el número de instantes, que es lo que permite llegar a 150 s de contexto. **Y lo comprueban** (Apéndice C): repiten todo a 100 Hz, que sí cumple Nyquist.

Como eso duplica los instantes, hay que bajar el contexto de 150 s a 75 s por memoria de GPU.

Modelo	13 % de los datos			100 % de los datos		
	MEG-MASC	Armeni	LibriBrain	MEG-MASC	Armeni	LibriBrain
MEG-XL (50 Hz, 150 s)	47,0	54,9	57,3	46,4	61,2	63,0
MEG-XL (100 Hz, 75 s)	41,9	51,7	52,2	41,8	56,7	58,6

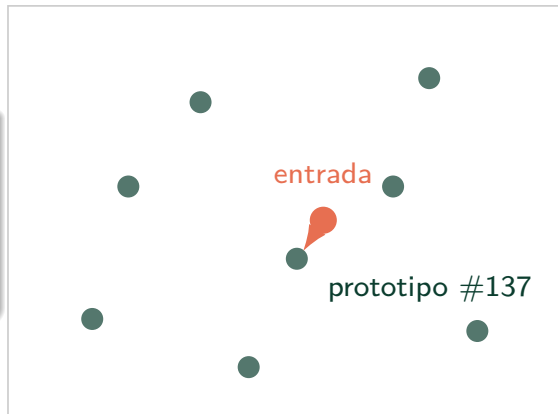
Lectura: el contexto largo compensa el *aliasing*. Pero es una compensación empírica, no una justificación teórica.

Paso 1 — Cuantización vectorial: un diccionario de formas de onda

Cuantización vectorial (VQ)

- 1 Tienes un **diccionario** (*codebook*) de V vectores prototipo, aprendidos previamente.
- 2 Llega un vector de entrada.
- 3 Buscas el prototipo **más parecido**.
- 4 Transmites sólo **su índice**.

Es exactamente la idea de los cuantificadores vectoriales de los códecs de voz clásicos. Aquí, $V = 256$ prototipos, es decir, 8 bits por símbolo.



espacio de características

Pérdida inevitable: sustituyes el vector real por el prototipo más cercano. El error residual es lo que motiva el siguiente paso.

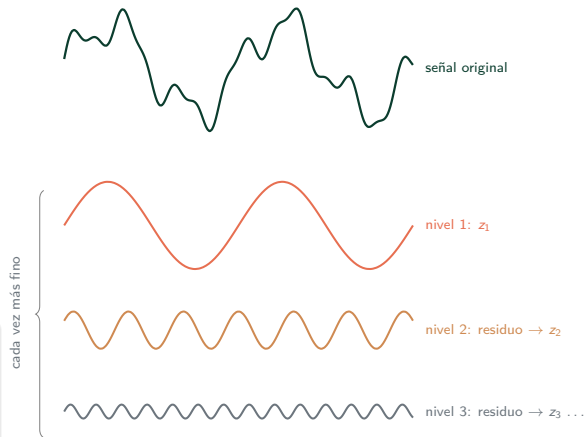
Paso 1 — RVQ: cuantizar el error, seis veces

Cuantización vectorial residual (RVQ). Es una **aproximación sucesiva**, igual que un ADC de aproximaciones sucesivas:

- 1 Nivel 1: aproxima la señal con el diccionario 1. Índice z_1 .
- 2 Calcula el **residuo** = señal - aproximación.
- 3 Nivel 2: aproxima **el residuo** con el diccionario 2. Índice z_2 .
- 4 ...y así hasta $Q = 6$ niveles.

Por qué es una buena idea

Con 6 niveles de 256 entradas se cubren $256^6 \approx 2,8 \times 10^{14}$ combinaciones, con sólo 6×256 prototipos que almacenar. Un único diccionario equivalente sería inmanejable.



Paso 1 — BioCodec aplicado al MEG: las cifras exactas

- El tokenizador es **BioCodec** (Avramidis et al., 2025), entrenado con miles de horas de **EEG a 250 Hz**.
- Se aplica **canal a canal, de forma independiente**: no mezcla sensores.
- Está **congelado**: sus pesos no se tocan durante todo el entrenamiento de MEG-XL. Sólo produce etiquetas.
- Factor de compresión temporal: $r = 12$.

Con números concretos

Sensores	$C = 306$
Duración	150 s a 50 Hz
Muestras	$T = 7\,500$
Tras comprimir	$T' = 625$
Resolución del token	$12/50 = 0,24$ s
Posiciones	$306 \times 625 = 191\,250$
Tokens totales	$191\,250 \times 6 \approx 1,15$ M

La transformación

$$X \in \mathbb{R}^{C \times T} \longrightarrow Z \in \{0, \dots, 255\}^{C \times T' \times Q}$$

con $T' = T/12$, $Q = 6$, $V = 256$.

Doble función del tokenizador: comprime la entrada $12\times$ y fabrica las etiquetas del entrenamiento autosupervisado.

Cada token resume 240 ms de un único sensor. Una ventana de palabra de 3 s son unos 12–13 pasos de token.

Paso 1 — Dos decisiones que merecen discusión

Un tokenizador de EEG, sobre MEG

BioCodec se entrenó con EEG a 250 Hz. Aquí se usa sobre MEG a 50 Hz. **Funciona:** lo eligen frente a BrainTokenizer (entrenado con MEG) porque reconstruye mejor. Error de reconstrucción: **0,41** (BioCodec) frente a 0,69 (BrainTokenizer).

Es también la razón por la que el salto a EEG del TFM es razonable: parte del sistema *ya viene* de EEG.

Consecuencia arquitectónica directa: como el eje de sensores **no** está comprimido, hay 306 posiciones espaciales por instante. Eso es justo lo que hace inviable la atención completa.

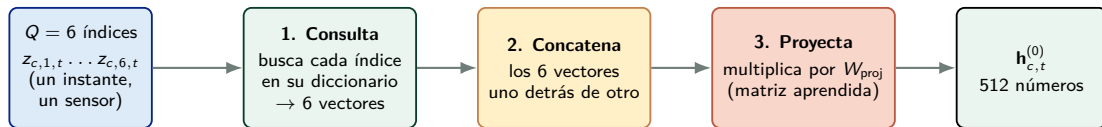
No comprimir el eje de sensores

BrainTokenizer comprime también entre canales; BioCodec no. Eso produce **más tokens** (peor para la memoria), pero **pierde menos información**.

Razonamiento explícito de los autores: **aún no sabemos qué partes de la señal son relevantes** para decodificar habla, así que conviene no descartar nada en el tokenizador y dejar que el transformer aprenda las relaciones entre canales.

Paso 2 — De índices a vectores

Un índice (“prototipo #137”) no se puede meter en una red: no hay ninguna relación numérica sensata entre el 137 y el 138. Hay que convertirlo en un vector.



$$\mathbf{h}_{c,t}^{(0)} = W_{\text{proj}} \left[\mathbf{e}_{z_{c,1,t}}^{(1)} ; \dots ; \mathbf{e}_{z_{c,Q,t}}^{(Q)} \right]$$

Los diccionarios $\mathbf{e}^{(q)}$ están **congelados** (son de BioCodec). La matriz W_{proj} **sí se aprende**.

Paso 2 — El modelo no sabe dónde está cada sensor

El problema

Hasta ahora, $\mathbf{h}_{c,t}^{(0)}$ sólo describe **la forma de onda**. El modelo no tiene ni idea de si el canal c está sobre el lóbulo temporal izquierdo o sobre el occipital. Y distintos equipos MEG tienen distinto número y disposición de sensores.

Solución: describir cada sensor por lo que físicamente es. Tres piezas de información que ya están en el fichero de la grabación:

Posición $\mathbf{p}_c \in \mathbb{R}^3$

Coordenadas 3D de la bobina en el casco.

Orientación $\mathbf{o}_c \in \mathbb{R}^3$

Hacia dónde apunta. Determina qué componente del campo mide.

Tipo de sensor

Magnetómetro o gradiómetro. Categoría discreta.

Ventaja clave: así el modelo puede entrenarse con **varios sistemas MEG distintos sin cambiar la arquitectura**. No aprende “el canal 42”, aprende “un sensor situado aquí, orientado así”.

Paso 2 — Por qué las coordenadas no se meten tal cual

Meter (x, y, z) directamente funciona mal: las redes densas tienen un sesgo fuerte hacia funciones **suaves** y no distinguen bien sensores cercanos. La solución son las **características de Fourier gaussianas**:

$$\gamma(\mathbf{v}) = [\cos(2\pi B\mathbf{v}), \sin(2\pi B\mathbf{v})]$$

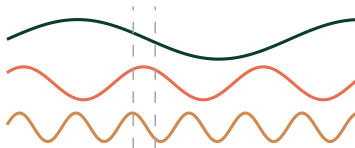
- B es una matriz de **frecuencias espaciales aleatorias**, sorteadas de $\mathcal{N}(0, \sigma^2)$ y **fijas** después.
- Es una descomposición en senos y cosenos de muchas frecuencias: convierte una posición en un patrón rico.
- Dos sensores próximos producen patrones distinguibles en las componentes de alta frecuencia.

entrada cruda x



señal pobre: casi lineal

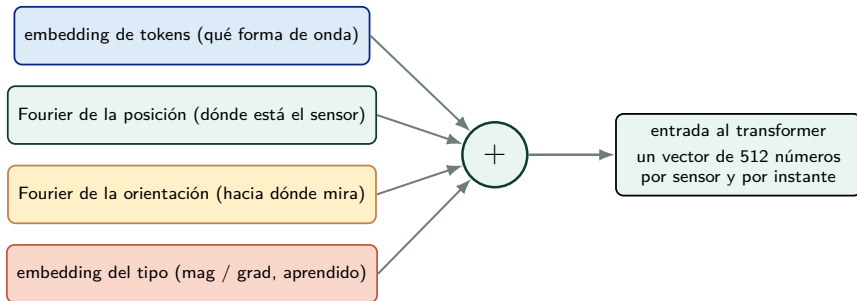
$\gamma(x)$: **varias frecuencias**



cada posición produce una firma de sensores
dos sensores próximos, aún distinguibles

$$d_{\text{fourier}} = 256; \quad \sigma_{\text{posición}} = 1,8; \quad \sigma_{\text{orientación}} = 1,0.$$

Paso 2 — La suma final



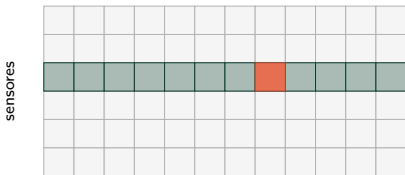
las tres últimas se proyectan antes a 512 dimensiones

- Se **suman**, no se concatenan: así la dimensión no crece y cada aportación se superpone sobre la misma representación.
- Resultado: un tensor $H^{(0)}$ de tamaño $306 \times 625 \times 512$. Unos **98 millones de números** por muestra de 2,5 minutos.

Paso 3 — La idea de la atención *criss-cross*

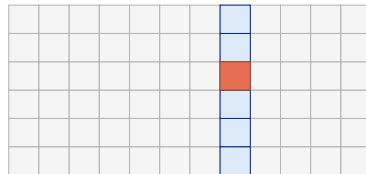
Recordatorio: atención completa sobre $306 \times 625 = 191\,250$ posiciones es imposible. La solución es **no mirar todos los pares**, sino sólo los pares que comparten fila o columna.

Atención temporal



un sensor mira su propio pasado y futuro
(625 posiciones)
tiempo

Atención espacial



un instante mira todos los sensores a la vez
(306 posiciones)
tiempo

Con dos capas seguidas, la información de **cualquier** sensor en **cualquier** instante puede llegar a cualquier otro: primero se propaga por la fila, luego por la columna. El nombre viene de esa cruz.

Paso 3 — Cómo se combinan las dos atenciones

MEG-XL no las aplica en serie, sino **en paralelo, partiendo el vector de características por la mitad**:

$$H^{(\ell)} = H^{(\ell-1)} + \left[\underbrace{\text{AtenciónEspacial}\left(H_{:d/2}^{(\ell-1)}\right)}_{\text{primeras 256 componentes}} ; \underbrace{\text{AtenciónTemporal}\left(H_{d/2:}^{(\ell-1)}\right)}_{\text{últimas 256 componentes}} \right]$$

Lectura de la fórmula, término a término

- Las 512 componentes se cortan en dos mitades de 256.
- La primera mitad se procesa **sólo** con atención entre sensores.
- La segunda mitad, **sólo** con atención a lo largo del tiempo.
- Se vuelven a pegar (concatenación) y se **suman a la entrada**: es la conexión residual.

Consecuencias prácticas

- Las dos atenciones se calculan **a la vez**: mitad del coste de hacerlas en serie.
- Son independientes dentro de una capa; la mezcla entre ambas mitades la hace la **red densa** del bloque, que sí ve las 512 componentes.
- RoPE se aplica sólo en la rama temporal.

Paso 3 — El ahorro, con los números encima de la mesa

Atención completa

$$\begin{aligned}\mathcal{O}((C \cdot T')^2) &= (306 \cdot 625)^2 \\ &= 191\,250^2 \approx 3,66 \times 10^{10}\end{aligned}$$

Atención *criss-cross*

$$\begin{aligned}\mathcal{O}(C \cdot T'^2 + T' \cdot C^2) \\ &= 306 \cdot 625^2 + 625 \cdot 306^2 \approx 1,78 \times 10^8\end{aligned}$$

$\approx 205 \times$

menos pares que calcular y que almacenar

Lo que se pierde

Dentro de **una sola capa**, la posición (sensor 12, instante 300) no puede mirar directamente a (sensor 200, instante 10). Necesita dos saltos. Con 8 capas hay de sobra, y el artículo muestra experimentalmente que la jerarquía resultante (local abajo, global arriba) es precisamente lo que hace útil al modelo.

Arquitectura

Capas L	8
Dimensión d_{model}	512
Cabezas de atención	8
Normalización	RMSNorm, pre-LN
Atención	<i>criss-cross</i>
Red densa	expansión $4\times$, SELU
Posición temporal	RoPE
Parámetros	$\approx 20\text{ M}$

Detalle que resuelve un problema real

Distintos sistemas MEG tienen distinto número de sensores. MEG-XL **rellena** hasta un número máximo de canales y usa una **máscara de sensores** que excluye los canales de relleno:

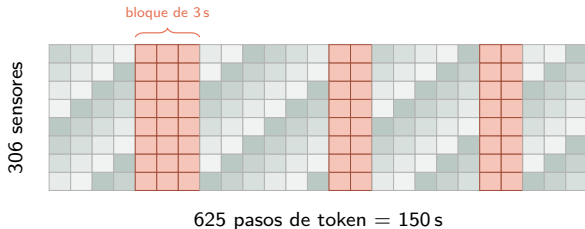
- del cálculo de la atención, y
- del cálculo de la pérdida.

Gracias a eso se puede entrenar con CamCAN, MOUS y SMN4Lang **mezclados**, sin tocar la arquitectura.

Esta misma máscara es la pieza que el TFM reutiliza para montajes EEG heterogéneos.

Paso 4 — El enmascaramiento, con todos sus porqués

- Se eligen al azar **bloques contiguos de 3 s** y se tapan.
- Se tapan **20 bloques**, hasta cubrir el **40 %** de la secuencia.
- La máscara se aplica **igual en todos los sensores** en esos instantes.
- Los tokens tapados se sustituyen por un **vector de máscara aprendido**, el mismo para todos.
- La pérdida se calcula **sólo sobre las posiciones tapadas**.



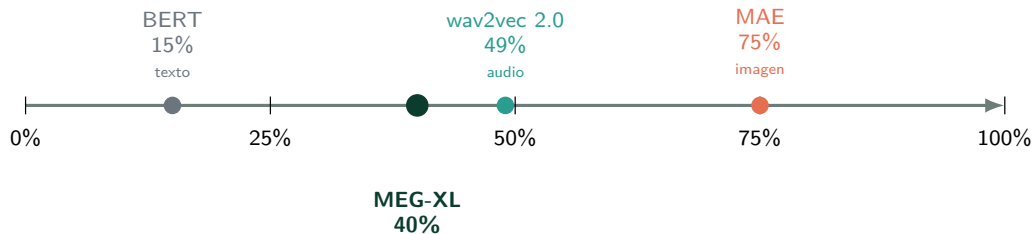
Las dos decisiones de diseño, explicadas

¿Por qué bloques largos y no muestras sueltas? Porque el MEG está fuertemente autocorrelado: tapar una muestra aislada se resuelve interpolando entre vecinas y no se aprende nada. Además, 3 s es la duración típica de la respuesta neural decodificable a una palabra.

¿Por qué la misma máscara en todos los sensores? Porque si se tapara un sensor y no sus vecinos, el modelo copiaría del sensor de al lado en el mismo instante. Tapando la columna entera, la única salida es **usar el contexto temporal**, que es justo lo que se quiere enseñar.

Paso 4 — ¿Por qué justo el 40 %?

Es un hiperparámetro ajustado a mano, pero el artículo lo sitúa en su contexto, y es informativo:



- **Texto (15 %)**: cada palabra lleva mucha información. Tapar mucho hace la tarea imposible.
- **Imagen (75 %)**: los píxeles vecinos son casi idénticos. Hay que tapar muchísimo para que la tarea no sea trivial.
- **Audio y MEG (~40–50 %)**: señales muy redundantes en el tiempo, pero no tanto como una imagen. MEG-XL cae donde cabía esperar, junto a un modelo de habla.

Paso 5 — Predicción y pérdida

Para cada posición tapada (c, t) , se toma la salida de la última capa $\mathbf{h}_{c,t}^{(L)}$ (512 números) y se predicen **los 6 índices** de golpe, con 6 clasificadores lineales independientes:

$$p(z_{c,q,t} \mid X_{\setminus M}) = \text{softmax}(W_q \mathbf{h}_{c,t}^{(L)}), \quad q = 1, \dots, 6$$

Cada W_q produce **256 números**, uno por prototipo del diccionario q ; la *softmax* los convierte en probabilidades que suman 1. La

pérdida es la **entropía cruzada** promediada sobre todas las posiciones tapadas, todos los sensores y los 6 niveles:

$$\mathcal{L} = -\frac{1}{|M| C Q} \sum_{t \in M} \sum_{c=1}^C \sum_{q=1}^Q \log p(z_{c,t,q} \mid X_{\setminus M})$$

Qué significa esa fórmula

“Mira la probabilidad que el modelo asignó al índice **correcto**; si es alta, el logaritmo es casi 0 y no penaliza; si es baja, el $-\log$ se dispara.” El nivel de azar es $1/256 \approx 0,4\%$.

Interpretación

Es el mismo tipo de pérdida que usarías para clasificar época por época en un clasificador de estadios de sueño. Aquí sólo hay muchísimas más clasificaciones simultáneas.

Resumen: las formas del tensor en cada paso

Etapa	Forma	Qué es cada número
MEG crudo	$306 \times 150\,000$	un valor de campo magnético
Tras preproceso (50 Hz)	$306 \times 7\,500$	idem, filtrado y estandarizado
Tras BioCodec	$306 \times 625 \times 6$	un índice entero de 0 a 255
Tras embeddings	$306 \times 625 \times 512$	una característica aprendida
Tras las 8 capas	$306 \times 625 \times 512$	idem, ya contextualizada
Salida del cabezal	$306 \times 625 \times 6 \times 256$	probabilidad de cada prototipo

0,24 s

cubre cada paso de token

191 250

posiciones de entrada

40 %

de la secuencia tapada

Hiperparámetros del preentrenamiento

Datos

Frecuencia original	1000 Hz
Remuestreo	50 Hz
Paso bajo / paso alto	40 Hz / 0,1 Hz
Estandarización	IQR = $[-1, 1]$ por 3 s
Línea base	$[0,0; 0,5]$ s por 3 s
Recorte	$[-5, 5]$
Longitud de muestra	150 s

Corpus de preentrenamiento

CamCAN	~700 suj.	reposo, sensorimotor
MOUS	104 suj.	escucha (neerlandés)
SMN4Lang	12 suj.	escucha (mandarín)

≈300 horas, >800 sujetos

Optimización

Pasos de entrenamiento	35 000
Tamaño de lote	1
Tasa de aprendizaje	10^{-4}
Decaimiento de pesos	10^{-4}
Calentamiento	250 pasos
Recorte de gradiente	1,0
Optimizador	AdamW
Pérdida	entropía cruzada

Tamaño de lote = 1

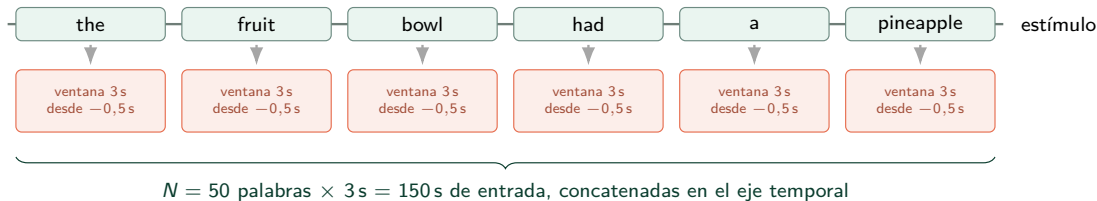
Una única muestra de 2,5 minutos ya ocupa toda la GPU.
El “lote” efectivo son los ≈1,15 M de tokens que contiene.

Preentrenamiento: ≈12 h en una H100.

- 1 El problema: leer palabras del cerebro
- 2 Vocabulario mínimo: qué es un transformer
- 3 MEG-XL paso a paso
- 4 De la señal a la palabra: ajuste y resultados**
- 5 Transferencia a EEG: el trabajo propio
- 6 Cierre

La tarea: decodificación contextual de palabras

Se sigue exactamente el protocolo de d'Ascoli et al. (2025), para que la comparación sea limpia.



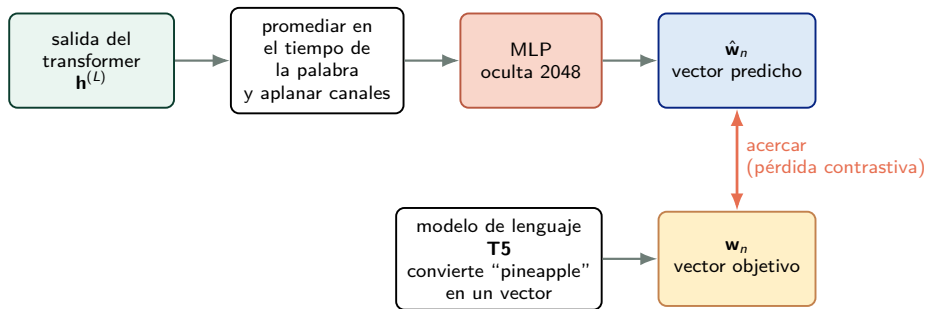
- Cada ventana empieza **0,5 s antes** del inicio de la palabra y dura **3 s**.
- Se concatenan 50 ventanas → **exactamente 150 s**, la misma longitud del preentrenamiento. Por eso no hace falta tocar la arquitectura.

Honestidad sobre el montaje

Esa entrada **no es continua en tiempo fisiológico**: se pegan trozos. Se hace así para replicar exactamente la línea base y para eliminar el efecto de la velocidad variable del habla.

¿Contra qué se compara la salida? Un embedding lingüístico

El modelo **no clasifica entre 50 palabras**. Predice un **vector que describe el significado** de la palabra, y luego se busca la palabra más parecida.



- **T5** es un modelo de lenguaje preentrenado; se usa sólo como **traductor de palabra a vector**, congelado. Palabras de significado parecido dan vectores parecidos.
- Ventaja sobre clasificar directamente: el modelo puede acertar "parcialmente" y, en principio, generalizar a palabras que no vio.

La pérdida contrastiva, sin jerga

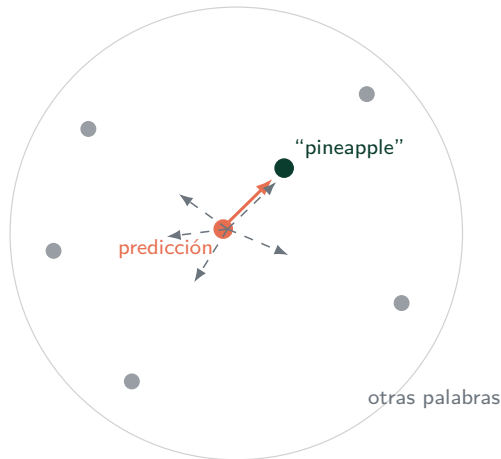
Dentro de cada lote de 50 palabras, la pérdida (**SigLIP**, variante D-SigLIP) hace dos cosas a la vez:

- **Acercar** el vector predicho al vector de *su* palabra.
- **Alejarlo** de los vectores de las otras 49 palabras del lote.

Detalle fino: la variante usada **enmascara las palabras repetidas** dentro del lote, para no penalizar por “alejarse” de un vector que en realidad es el correcto.

Por qué no basta con minimizar la distancia

Si sólo se acercara, el modelo podría colapsar y predecir siempre el mismo vector medio. El empuje hacia fuera es lo que obliga a **discriminar**.



Cómo se decide la palabra

Se toma el vector predicho $\hat{\mathbf{w}}_n$ y se compara con el vector de cada palabra del **vocabulario de recuperación**:

$$\hat{y}_n = \arg \max_y \cos(\hat{\mathbf{w}}_n, \mathbf{w}_y)$$

Es una búsqueda del **vecino más próximo** por similitud coseno. Nada más.

Vocabulario: las 50 palabras más frecuentes (y, en el apéndice, las 250 más frecuentes).

Al leer cualquier número de las tablas siguientes, el azar es 20 %. Un 47 % no es “poco”: es más del doble del azar en una tarea con SNR pésima.

Métrica: Top-10 balanceada

- **Top-10**: se cuenta acierto si la palabra correcta está entre las 10 más parecidas.
- **Balanceada**: macro-media **por clase de palabra**, no media sobre ejemplos.

Por qué balanceada: sin ello, acertar siempre “the” y “a” infla la cifra. El nivel de azar queda en $10/50 = 20\%$ (o $10/250 = 4\%$).

Hiperparámetros del ajuste fino

Ajuste

Dimensión oculta del MLP	2048
Épocas máximas	50
Parada temprana (paciencia)	10 épocas
Criterio de parada	Top-10 bal. en validación
Tamaño de lote	50 palabras
LR del transformer	10^{-5}
LR del MLP	10^{-3}
Decaimiento de pesos	10^{-4}
Optimizador	AdamW
Pérdida	D-SigLIP

Un detalle revelador

El transformer se ajusta con una tasa **100 veces menor** que el MLP. Razón: el transformer ya **sabe algo** y sólo hay que retocarlo con cuidado para no borrar lo aprendido; el MLP empieza de cero y necesita moverse rápido.

Datos de evaluación

Conjunto	Suj.	h/suj.	h total
MEG-MASC	27	2	54
Armeni	3	10	30
LibriBrain	1	52	52

Particiones 80:10:10, con **el mismo estímulo siempre en la misma partición.**

Resultado 1 — Contra otros modelos fundacionales

Modelo	Par.	Con el 13 % de los datos			Con el 100 % de los datos		
		MEG-MASC	Armeni	LibriBrain	MEG-MASC	Armeni	LibriBrain
BioCodec	1,0 M	19,8	20,0	19,9	31,2	37,1	41,9
EEGPT	4,7 M	19,6	20,3	20,3	26,3	20,8	22,9
BIOT	3,2 M	20,0	20,2	20,6	31,3	35,7	45,6
BBL	15 M	21,5	22,3	32,1	35,9	39,1	49,9
BrainOmni	8,4 M	18,7	21,0	29,7	19,1	62,3	63,0
LaBraM	5,8 M	33,2	26,3	40,3	31,1	42,0	47,7
MEG-XL	20 M	47,0	54,9	57,3	46,4	61,2	63,0
<i>Azar</i>	—	<i>20,0</i>	<i>20,0</i>	<i>20,0</i>	<i>20,0</i>	<i>20,0</i>	<i>20,0</i>

Lo importante está en la mitad izquierda

Con pocos datos, **casi todos los demás modelos se quedan en el azar** (19–21 %). MEG-XL llega a 47–57 %. Ese es el régimen clínicamente relevante.

Y un matiz honesto

BrainOmni gana en Armeni con todos los datos (62,3 frente a 61,2) pero se hunde en MEG-MASC (19,1). Es decir: funciona con pocos sujetos y muchas horas, no al revés.

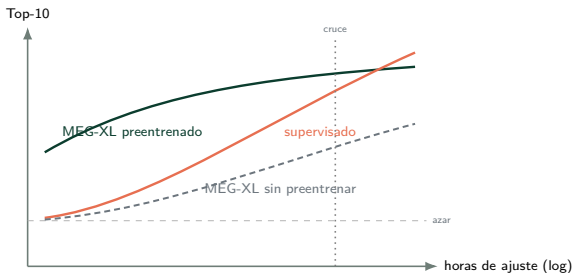
Resultado 2 — Eficiencia de datos frente al supervisado

- **MEG-MASC** (27 sujetos, 2 h cada uno): MEG-XL mejora al supervisado hasta en un 25 % y la ventaja **se mantiene con todos los datos**.
- **Armeni** (3 sujetos, 10 h): unos 10 puntos por encima hasta las ~15 h.
- **LibriBrain** (1 sujeto, 52 h): empatan hasta las 2,5 h; a partir de ahí **gana el supervisado**.

La regla que emerge

El preentrenamiento sustituye datos **del sujeto**, no datos en general. Muchos sujetos poco tiempo → gana preentrenar. Un sujeto muchísimo tiempo → gana entrenar desde cero.

El control más importante: MEG-XL *sin* preentrenar va muy por debajo. La ganancia viene del preentrenamiento, no de la arquitectura.



Esquema cualitativo de la Figura 3.

Resultado 3 — Más contexto, mejores representaciones

Experimento: preentrenar varios modelos con contextos de 15, 30, 45... 150 s; **congelarlos**; y entrenar encima sólo un clasificador lineal (**sonda lineal** o *linear probe*).

Por qué congelar

Si se ajustara todo el modelo, no se sabría si mejora por la representación aprendida o porque el ajuste la rehace. Con el modelo congelado, **lo único que se mide es la calidad de la representación**.

- Más contexto de preentrenamiento → mejores representaciones, con **rendimientos decrecientes a partir de ~100 s**.
- Se comparan dos condiciones: dar a todos 150 s en inferencia, o dar a cada uno su propia longitud. **Apenas hay diferencia**.

La conclusión menos obvia

Dar contexto largo **en inferencia** no sirve de nada si el modelo no fue **preentrenado** con contexto largo. Usar contexto largo es una **habilidad aprendida**, no una propiedad de la arquitectura. Es el mismo fenómeno de “generalización en longitud” conocido en modelos de lenguaje.

En predicción **cero-disparo** de señal enmascarada sobre conjuntos nunca vistos, la mejora **no satura** ni siquiera a 150s: sólo para porque no cabe más en la VRAM.

Resultado 4 — ¿Qué aprende exactamente el modelo?

Se analizan los pesos de atención temporal A con dos medidas:

$$\text{MAD}(t) = \sum_k A_{t,k} |t - k|$$

$$H(t) = - \sum_k A_{t,k} \log A_{t,k}$$

Distancia media de atención (MAD)

A qué distancia temporal mira cada capa, en promedio. Se convierte a segundos multiplicando por $r/f = 12/50 = 0,24$.

Entropía de la atención

Cuánto se reparten los pesos. Entropía **alta** = mira a todas partes por igual (difuso). Entropía **baja** = selecciona unos pocos instantes.

Los dos hallazgos

- 1 **Jerarquía.** Los modelos de contexto largo miran **cerca en las capas bajas** y van ampliando el alcance con la profundidad. Los de contexto corto miran difusamente desde la primera capa.
- 2 **Selectividad.** La entropía **baja** al aumentar el contexto de preentrenamiento: el modelo aprende a elegir.

*“Lo que el contexto largo enseña no es la capacidad de mirar lejos, sino **cuándo** merece la pena hacerlo.”*

Del método

- Habla **percibida**, no imaginada. El caso clínico real es el segundo.
- Vocabulario de **50 palabras** (250 en el apéndice, con caída clara de rendimiento).
- Sujetos **sanos**, conjuntos públicos.
- La entrada de ajuste no es fisiológicamente continua.
- El preprocesado incumple Nyquist (compensado empíricamente).

Del alcance

- MEG es un aparato **grande, caro y fijo**. No es una solución clínica portátil.
- Se caracterizan los patrones de atención, pero **no se sabe qué estructura neural concreta** han capturado.
- La comparación con contexto >150 s no se puede hacer: no hay VRAM.

Y de ahí sale la pregunta siguiente

Si MEG no es desplegable, ¿se puede llevar todo esto a EEG?

- 1 El problema: leer palabras del cerebro
- 2 Vocabulario mínimo: qué es un transformer
- 3 MEG-XL paso a paso
- 4 De la señal a la palabra: ajuste y resultados
- 5 Transferencia a EEG: el trabajo propio**
- 6 Cierre

Por qué EEG, y qué se rompe al cambiar de modalidad

Motivación

EEG es **portátil, barato y clínicamente desplegable**. MEG no. Si las representaciones aprendidas en MEG sirven en EEG, el camino hacia una BCI real se acorta mucho.

Hay un punto a favor de partida: el tokenizador BioCodec **ya se entrenó con EEG**.

	MEG	EEG
Qué mide	campo magnético	potencial eléctrico
Canales	~306	32–128 típicamente
Tipo de sensor	mag / grad	un solo tipo
Orientación	bien definida	no existe
Posiciones	fijas por el casco	variables por montaje
Distorsión del cráneo	baja	alta (difusión)

Los tres agujeros concretos en la arquitectura de MEG-XL

1. El embedding de **orientación** no tiene equivalente en EEG.
2. El embedding de **tipo de sensor** sólo conoce “mag” y “grad”.
3. Los montajes EEG son **heterogéneos** entre conjuntos de datos.

Qué se conserva y qué se adapta

Se conserva intacto

Tokenizador BioCodec (congelado)

Atención *criss-cross*, 8 capas

RMSNorm, residuales, SELU, RoPE

Objetivo de tokens enmascarados

Ajuste contrastivo contra T5



Se adapta

Máscaras de sensores por montaje

Codificación explícita del tipo de canal

Posiciones 3D del montaje EEG

Adaptador de datos y alineamiento

Preentrenamiento continuo con EEG

La máquina de MEG-XL se mantiene; lo que cambia es **cómo se le describe un sensor y con qué se le alimenta.**

El adaptador de datos EEG

Preprocesado de la señal

- 1 Remuestreo a la frecuencia común del modelo.
- 2 Normalización con el mismo esquema robusto por subsegmentos.
- 3 Representación común de sensores: posición 3D del montaje y **codificación explícita del tipo de canal**.
- 4 **Máscara de sensores** para cubrir montajes con distinto número de electrodos.

Alineamiento con las palabras

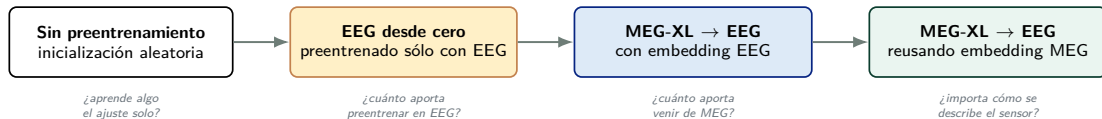
- 1 Leer las anotaciones de estímulo del conjunto (inicio de cada palabra).
- 2 Extraer la ventana temporal correspondiente a cada palabra.
- 3 Concatenar ventanas para formar la secuencia larga que espera el modelo.

Y para el preentrenamiento

No hacen falta etiquetas: se construyen **flujos continuos** por sujeto y sesión, y se cortan en ventanas de 150 s.

Conjuntos usados en el preentrenamiento EEG: **escucha** (OpenNeuro ds004408, ds007808, SparrKULee) y **lectura** (EEGDash NM000228, ZuCo 2.0).

El diseño experimental: aislar cada efecto



- Ajuste supervisado y evaluación final siempre sobre **OpenNeuro ds004408**, reservado y no usado en preentrenamiento.
- Métrica principal: **Top-10 balanceada con vocabulario de 250 palabras** (azar = 4 %).
- Particiones deterministas por sujeto; una única semilla (limitación conocida).

Resultados de la transferencia a EEG

Condición	Ép.	Top-10, 50	Bal., 50	Bal., 250	Bal./azar
Sin preentrenamiento	3	27,77 %	20,00 %	4,05 %	1,01
Preentrenado sólo con EEG	5	64,62 %	61,63 %	19,95 %	4,99
MEG-XL con embedding EEG	4	63,62 %	62,50 %	20,56 %	5,14
MEG-XL con embedding MEG	3	64,92 %	64,19 %	22,32 %	5,58
<i>Azar uniforme</i>	—	20,00 %	20,00 %	4,00 %	1,00

1. El ajuste solo no basta

Sin preentrenar: 4,05 % balanceado, es decir, **el azar**. El 14,66 % sin balancear era un espejismo de palabras frecuentes.

2. Preentrenar es el salto grande

De 4,05 % a 19,95 %. Casi toda la ganancia viene de haber preentrenado, aunque sea sólo con EEG.

3. Venir de MEG añade más

Hasta 22,32 %, **5,6 veces el azar**. Ganancia menor que la anterior, pero consistente.

Dos observaciones que conviene mirar de cerca

El mejor checkpoint aparece en la época 3

- A partir de ahí la validación deja de mejorar **aunque la pérdida de entrenamiento siga bajando**: sobreajuste de manual.
- Lectura positiva: el modelo **aprovecha muy pronto** lo transferido.
- Lectura prudente: con EEG, prolongar el ajuste **degrada** la generalización antes de mejorarla.

Reusar el embedding de magnetómetro gana

Resultado contraintuitivo: describir los electrodos EEG con el embedding de **magnetómetro** de MEG-XL va mejor que crear un embedding EEG nuevo. Explicación plausible:

reutilizar el embedding preentrenado **preserva la geometría** del espacio de representación aprendido; un embedding nuevo se inicializa al azar y hay que aprenderlo con muy pocos datos. **Con una sola semilla, conviene no sobreinterpretarlo.**

En preentrenamiento autosupervisado las tres variantes son casi idénticas (diferencias $<0,001$ en la pérdida de escucha). **La diferencia sólo aparece en la tarea final.**

Situación frente a trabajos previos

Referencia más cercana

Jayalath et al. sobre el conjunto de Broderick (equivalente a ds004408): entrenando sólo con Broderick se quedan **alrededor del azar**; combinando con LibriBrain llegan a $\approx 7\%$.

22,32 %

Top-10 balanceada, 250 palabras
(azar: 4 %)

Cautelas que hay que decir en voz alta

- Las **particiones no son idénticas** entre trabajos.
- El preprocesado difiere entre implementaciones.
- **Una sola semilla** en este experimento.
- No se ha hecho todavía el barrido reduciendo el porcentaje de datos supervisados, así que **no se puede afirmar** que necesite menos horas etiquetadas.

Lo que sí se puede afirmar: una arquitectura de contexto largo transferida desde MEG alcanza en EEG un resultado **claramente por encima del azar y del mismo orden que los mejores publicados** para este conjunto.

Consolidar lo que hay

- Más **semillas** y más particiones: confirmar que las diferencias entre condiciones son reales.
- Barridos con **menos datos supervisados** de ds004408, para medir de verdad la eficiencia de adaptación.
- Más **tokenizadores** además de BioCodec.

Ampliar

- Más **bandas de frecuencia** y frecuencias de remuestreo. Aquí entra una consideración práctica: aunque el límite teórico es $f_s/2$, en la práctica la información realmente utilizable llega hasta $\sim f_s/5$, y eso debería guiar la elección.
- **Embeddings separados por zona cerebral**.
- Encadenar un **modelo de lenguaje** para generación de texto libre.
- Probar con **grabaciones propias** de casco EEG.

- 1 El problema: leer palabras del cerebro
- 2 Vocabulario mínimo: qué es un transformer
- 3 MEG-XL paso a paso
- 4 De la señal a la palabra: ajuste y resultados
- 5 Transferencia a EEG: el trabajo propio
- 6 Cierre

Las cinco ideas que merece la pena llevarse

- ❶ **Un transformer es un promedio ponderado adaptativo apilado.** Los pesos se calculan a partir de los datos en cada posición y en cada capa. Todo lo demás (normalizaciones, residuales, red densa) es maquinaria para que apilarlo funcione.
- ❷ **El problema clínico no es la precisión máxima, es la eficiencia de datos.** Un paciente no puede dar 50 horas de grabación. Preentrenar con muchos sujetos sustituye a grabar mucho a uno solo.
- ❸ **Tokenizar una señal continua tiene dos propósitos:** dar un objetivo de clasificación bien definido y comprimir lo suficiente para que 2,5 minutos quepan en memoria.
- ❹ **La atención *criss-cross* es lo que hace viable el contexto largo:** factorizar en tiempo y sensores reduce el coste unas 205 veces, a cambio de necesitar dos capas para conectar cualquier par.
- ❺ **Usar contexto largo es una habilidad aprendida, no un regalo de la arquitectura.** Dar 150 s a un modelo preentrenado con 15 s no sirve de nada.

- **Embedding**: vector de números que representa un objeto.
- **Token**: símbolo de un catálogo finito; aquí, el índice de un prototipo del diccionario.
- **Codebook / diccionario**: conjunto de vectores prototipo de la cuantización.
- **RVQ**: cuantización por aproximaciones sucesivas del residuo; 6 niveles de 256 prototipos.
- **Atención**: promedio ponderado cuyos pesos se calculan de los datos.
- **Cabeza**: una de las 8 atenciones paralelas con criterios de similitud distintos.
- **Softmax**: exponencial normalizada; convierte números en probabilidades que suman 1.
- **Conexión residual**: salida = entrada + corrección aprendida.
- **RMSNorm**: normalización de amplitud por valor cuadrático medio.
- **RoPE**: codificación de posición por rotación; hace la atención sensible a la distancia relativa.
- **Preentrenamiento**: fase sin etiquetas, con muchos datos.
- **Ajuste fino**: fase con etiquetas, pocos datos, tasa de aprendizaje baja.
- **Sonda lineal**: clasificador lineal sobre un modelo congelado; mide la calidad de la representación.
- **Cero-disparo**: evaluar sin ajustar nada en el conjunto nuevo.

Referencias principales

- D. Jayalath, O. Parker Jones (2026). *MEG-XL: Data-Efficient Brain-to-Text via Long-Context Pre-Training*. arXiv:2602.02494.
- S. d'Ascoli et al. (2025). *Towards decoding individual words from non-invasive brain recordings*. Nature Communications 16.
- K. Avramidis et al. (2025). *Neural codecs as biosignal tokenizers* (BioCodec). arXiv:2510.09095.
- Q. Xiao et al. (2025). *BrainOmni: A brain foundation model for unified EEG and MEG signals*. NeurIPS.
- J. Wang et al. (2025). *CBraMod: A criss-cross brain foundation model for EEG decoding*. ICLR.
- A. Vaswani et al. (2017). *Attention is all you need*. NeurIPS.
- J. Devlin et al. (2019). *BERT*. NAACL-HLT. A. Baevski et al. (2020). *wav2vec 2.0*. NeurIPS.
- N. Zeghidour et al. (2022). *SoundStream*. IEEE/ACM TASLP. Z. Borsos et al. (2023). *SoundStorm*.
- M. Tancik et al. (2020). *Fourier features let networks learn high frequency functions*. NeurIPS.
- J. Su et al. (2024). *RoFormer: Enhanced transformer with rotary position embedding*. Neurocomputing.
- X. Zhai et al. (2023). *Sigmoid loss for language image pre-training* (SigLIP). ICCV.

Código y pesos de MEG-XL: <https://github.com/neural-processing-lab/MEG-XL> · Adaptación a EEG: <https://github.com/odraIA/EEG-XL>

Gracias

¿Preguntas?

Ricardo Díaz Peris · DSIC, Universitat Politècnica de València